

Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada Objetos
Segundo Ciclo, 2026
Catedrático: Erick Marroquín



Análisis de Laboratorio 1

Estudiantes:
Emily Lucila Menchú Menchú - 20472

Fecha: 10/08/2026

Análisis

Clases a definir:

- HabilidadEspecial
- Pokemon
- Jugador
- Ronda
- Combate
- VistaCombate
- ControladorCombate
- Main

Relaciones entre clases:

- Pokemon tiene una relación de composición con HabilidadEspecial, ya que cada Pokémon posee una habilidad propia.
- Jugador tiene una relación de agregación con Pokemon, ya que posee cuatro Pokémon que pueden existir independientemente.
- Combate tiene una relación de agregación con Jugador, ya que utiliza dos jugadores que existen independientemente del combate.
- Combate tiene una relación de composición con Ronda, ya que las cuatro rondas son parte del combate.
- Ronda tiene una relación de agregación con Pokemon, ya que utiliza un atacante y defensor pero que existen independientemente.
- ControladorCombate tiene una relación de agregación con Combate y VistaCombate, ya que los utiliza para coordinar el programa.
- Main tiene una relación de dependencia con ControladorCombate, ya que lo utiliza para iniciar el programa.

Tabla de Propiedades

#	Tipo de dato	Nombre	Visibilidad	Motivos	Clase
1	String	nombre	private	Para guardar el nombre de la habilidad	HabilidadEspecial
2	int	probabilidadActivación	private	Guarda el número de la probabilidad de que se active la habilidad especial	HabilidadEspecial
3	String	nombre	private	Guarda nombre del pokemon	Pokemon
4	int	tipo	private	Guarda un int que hará referencia a su tipo	Pokemon
5	int	ataque	private	Guarda la magnitud de su ataque de 0 a 100	Pokemon

6	int	defensa	private	Guarda la magnitud de defensa de 0 a 100	Pokemon
7	booleano	usado	private	Indica si el pokemon ya participó en una ronda	Pokemon
8	Habilidad Especial	habilidadEspecial	private	Guarda la habilidad especial	Pokemon
9	String	nombre	private	Guarda nombre	Jugador
10	Pokemon	pokemon1	private	Guarda primer pokemon	Jugador
11	Pokemon	pokemon2	private	Guarda segundo pokemon	Jugador
12	Pokemon	pokemon3	private	Guarda tercer pokemon	Jugador
13	Pokemon	pokemon4	private	Guarda cuarto pokemon	Jugador
14	int	rondasGanadas	private	Almacena las rondas ganadas del jugador	Jugador
15	Pokemon	atacante	private	Almacena el pokemon atacante	Ronda
16	Pokemon	defensor	private	Almacena el pokemon del segundo jugador	Ronda
17	Pokemon	ganador	private	Guarda el pokemon ganador de la ronda y null significa empate	Ronda
18	int	ataqueFinal	private	Indica el ataque final despues de aplicar habilidad especial y bonos	Ronda
19	int	defensaFinal	private	Indica la defensa final después de aplicar habilidad especial y bonos	Ronda
20	Ronda	ronda1	private	Guarda la ronda 1	Combate
21	Ronda	ronda2	private	Guarda la ronda 2	Combate
22	Ronda	ronda3	private	Guarda la ronda 3	Combate
23	Ronda	ronda4	private	Guarda la ronda 4	Combate
24	Jugador	jugador1	private	Guarda jugador 1	Combate
25	Jugador	jugador2	private	Guarda jugador 2	Combate
26	Jugador	ganador	private	Guarda el jugador ganador del combate y null representa empate	Combate

27	Combate	combate	private	Crea el combate	ControladorCombate
28	VistaCombate	vistaCombate	private	Para instanciar la vista del combate	ControladorCombate
29	Scanner	sc	public	Instanciar el scanner para ingresar datos	VistaCombate

Tabla de métodos

#	Tipo de retorno	Nombre	Visibilidad	Parámetro	Motivo	Clase
1	String	getNombre	public	-	Para obtener el nombre	HabilidadEspecial
2	int	getProbabilidadActivación	public	-	Para obtener la probabilidad de activación	HabilidadEspecial
3	boolean	seActiva	public	-	Hace el sorteo para determinar si se aplica habilidad especial	HabilidadEspecial
4	int	generarPotenciador	public	-	Genera un porcentaje aleatorio entre 0 y 25 para el potenciador.	HabilidadEspecial
5	-	HabilidadEspecial	public	nombre:String, probabilidad de activación: int	Constructor	HabilidadEspecial
6	String	getNombre	public	-	Para obtener nombre	Pokemon
7	int	getTipo	public	-	Para obtener tipo	Pokemon
8	int	getAtaque	public	-	Para obtener ataque incluyendo la activación de habilidad especial	Pokemon
9	int	getDefensa	public	-	Para obtener defensa incluyendo la activación de la habilidad especial	Pokemon
10	boolean	estaUsado	public	-	Para obtener activo	Pokemon

11	void	marcarComoUsado	public	-	Marca al pokemon cómo usado	Pokemon
12	Habilidad Especial	getHabilidad Especial	public	-	Para obtener habilidad especial	Pokemon
13	-	Pokemon	public	nombre:String, tipo:int, ataque:int, defensa:int, habilidad especial: HabilidadEspecial	Constructor de la clase pokémon	Pokemon
14	String	getNombre	public		Obtiene nombre	Jugador
15	Pokemon	seleccionarPokemon	public		Selecciona aleatoriamente a un pokemon que no haya sido usado	Jugador
16	int	getRondasGanadas	public		Obtiene rondas ganadas	Jugador
17	void	aumentarRondasGanadas	public		Suma 1 a las rondas ganadas	Jugador
18		Jugador	public	nombre:String, pokemon1:Pokemon, pokemon2:Pokemon, pokemon3:Pokemon, pokemon4:Pokemon		Jugador
19	-	Ronda	public	atacante:Pokemon, defensor:Pokemon	Constructor de la ronda	Ronda
20	void	iniciarBatalla	public	-	Ejecuta los cálculos necesarios y determina el ganador de la ronda	Ronda

21	int	calcularBonoTipo	private	-	Calcula el bono de tipo entre atacante y defensor	Ronda
22	int	calcularAtaqueEfectivo	private	-	Calcula el ataque del atacante considerando su posible potenciador	Ronda
23	int	calcularDefensaEfectiva	private	-	Calcula la defensa del defensor considerando su posible potenciador	Ronda
24	Pokemon	getGanador	public	-	Obtiene el pokemon ganador	Ronda
25	int	getAtaqueFinal	public	-	Obtiene la cantidad del ataque final	Ronda
26	int	getDefensaFinal	public	-	Obtiene la cantidad del defensa final	Ronda
27	int	getDiferencia	public	-	Obtiene la diferencia	Ronda
28	-	Combate	public	jugador1:Jugador, jugador2:Jugador	Constructor de la clase Combate	Combate
29	void	setRonda1	public	ronda:Ronda	Guarda la primera ronda realizada	Combate
30	void	setRonda2	public	ronda:Ronda	Guarda la segunda ronda realizada	Combate
31	void	setRonda3	public	ronda:Ronda	Guarda la tercera ronda realizada	Combate
32	void	setRonda4	public	ronda:Ronda	Guarda la cuarta ronda realizada	Combate
33	Jugador	determinarGanador	public	-	Compara las rondas ganadas y determina al ganador	Combate
34	Jugador	getGanador	public	-	Obtiene al jugador ganador o null si el combate terminó en empate	Combate

35	Jugador	getJugador1	public	-	Obtiene al primer jugador	Combate
36	Jugador	getJugador2	public	-	Obtiene al segundo jugador	Combate
37	-	Controlador Combate	public	-	Constructor del controlador	ControladorCombate
38	void	iniciarCombate	public	-	Coordina las cuatro rondas y alterna los roles de atacante y defensor	ControladorCombate
39	-	VistaCombate	public	-	Constructor de la vista	VistaCombate
40	void	mostrarInicioCombate	public	nombreJugador1:String, nombreJugador2:String	Muestra el inicio del combate y sus participantes	VistaCombate
41	void	mostrarRonda	public	numeroRonda:int, jugadorAtacante:String, atacante:String, , jugadorDefensor:String, defensor:String	Muestra el número de ronda y los Pokémon que participan	VistaCombate
42	void	mostrarResultadoRonda	public	habilidadAtacante:String, ataque:double, habilidadDefensor:String, defensa:double, diferencia:double, ganador:String	Muestra los cálculos y el resultado de una ronda	VistaCombate
43	void	mostrarGanadorCombate	public	ganador:String	Muestra el nombre del jugador ganador	VistaCombate

44	void	mostrarEmp ate	public	-	Informa que el combate terminó empatado	VistaCombate
45	String	pedirNombr eJugador	public	-	Pide nombre del jugador	VistaCombate
45	void	main	public	args: String[]	Punto de inicio del programa	Main